

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL3-Reverse Engineering

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192365045
Name	Shaik Mahaboob Basha

COURSE OUTCOME COVERED

CO5: Design intelligent agents and real-time AI-based systems (chatbots, expert systems, emotion detection, edge AI, autonomous drones, and related applications) by selecting appropriate agent architectures, knowledge representation, and reasoning mechanisms to solve real-world problems. (BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1. The following is a conversation between a user and an AI-based flight-booking chatbot. Study the transcript given below and reverse-engineer the chatbot's design by: (a) writing its PEAS description, (b) classifying the type of agent it represents with justification, (c) identifying the likely component (NLU, Dialogue Manager, Knowledge Base, or NLG) responsible for each chatbot response, and (d) suggesting one enhancement using a learning-agent architecture.

Speaker	Message
User	I want to book a flight to Chennai next Friday.
Chatbot	Sure! What time would you like to depart, and do you have a preferred airline?
User	Morning flight, no preference.
Chatbot	I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?

Q2. The block diagram below shows the processing pipeline of a Real-Time Emotion Detection System. Reverse-engineer the system by: (a) explaining the function performed at each stage of the pipeline, (b) identifying the likely AI/ML technique used at each stage, (c) writing the PEAS description of the system, and (d) classifying the

Real-Time Emotion Detection System — processing pipeline



agent type (e.g., simple reflex, model-based) with justification.

Q3. An Expert System Shell for medical diagnosis contains the rule base, input facts, and reasoning trace given below. Study it and reverse-engineer the shell by: (a) identifying the type of chaining (forward/backward) used to reach the diagnosis, with justification, (b) identifying the expert-system-shell components illustrated (knowledge base, inference engine, explanation facility), (c) explaining how the shell would perform knowledge acquisition to add a new rule for dengue, and (d) explaining, with reasoning, whether backward chaining could reach the same diagnosis.

Rule	IF (condition)	THEN (conclusion)
R1	fever = yes AND cough = yes	suspect = flu
R2	suspect = flu AND body_ache = yes	diagnosis = Influenza
R3	fever = yes AND rash = yes	suspect = measles

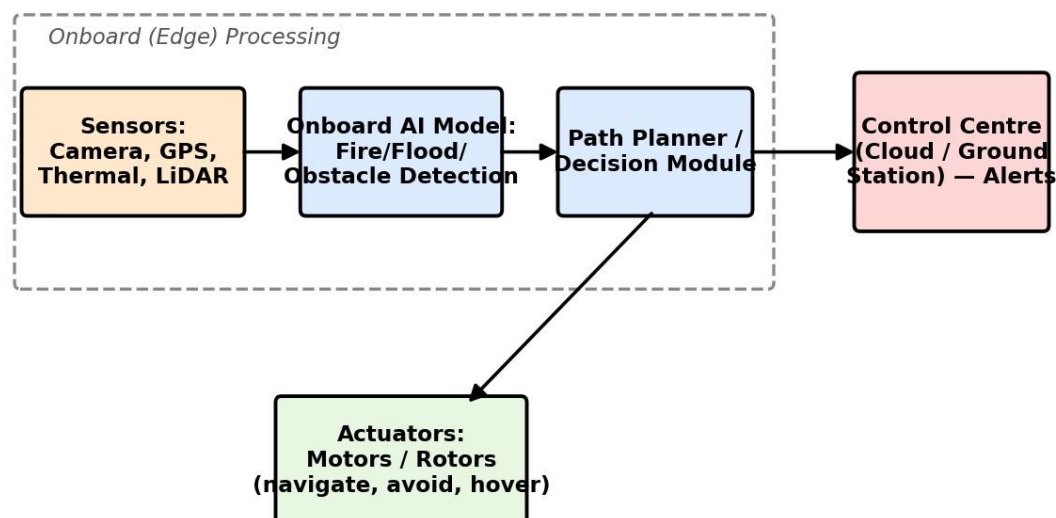
Given facts: fever = yes, cough = yes, body_ache = yes

Reasoning trace: R1 fires first (fever & cough matched) → suspect = flu. R2 then fires (suspect = flu & body_ache matched) → diagnosis = Influenza.

Explanation generated by the shell: “Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza.”

Q4. The diagram below shows the system architecture of an Autonomous Drone used for Disaster Detection with Edge AI deployment. Reverse-engineer the design by: (a) writing the PEAS description of the drone agent, (b) classifying the agent type (goal-based/utility-based) with justification, (c) identifying which components run on the edge versus the cloud and justifying why on-edge processing was chosen for detection and obstacle avoidance, and (d) identifying whether the agent's control architecture is reactive, deliberative, or hybrid, with justification.

Autonomous Drone — Disaster Detection (Edge AI) system architecture



Q5. The table below shows sample output logs from an AI-based Deepfake & Face Authenticity Verification system across five video frames. Study the output and reverse-engineer the system by: (a) identifying the likely input features used by the model (e.g., blink rate, texture/artifact score), (b) identifying the type of model/technique likely used to produce these outputs, (c) explaining how the final Real/Fake decision may have been derived from the frame-level confidence scores,

and (d) suggesting one way a swarm/ensemble-based approach could improve the system's reliability.

Frame No.	Blink Rate (per 10s)	Texture/Artifact Score	Fake Confidence (%)	Prediction
1	4	0.12	8	Real
2	1	0.61	74	Fake
3	0	0.83	91	Fake
4	5	0.09	5	Real
5	2	0.55	68	Fake

Overall system verdict: Fake (3 of 5 frames flagged — majority vote)

Code of Conduct:

I Shaik Mahaboob Basha(192365045) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Shaik Mahaboob Basha

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Identification of Agent Type & PEAS (correctly inferred from the given artifact)	Correctly identifies the agent type and gives a complete, accurate PEAS description, fully consistent with the given artifact.	Identifies the agent type and PEAS description correctly with only minor omissions.	Partially correct identification of agent type/PEAS; some elements are missing or inconsistent with the artifact.	Agent type/PEAS identification is largely incorrect or not attempted.	10
Identification of Architecture & Components (mapping artifact evidence to system components)	Accurately identifies all relevant architectural components and correctly maps each to specific evidence in the artifact.	Identifies most components correctly with reasonable mapping to the artifact; minor gaps present.	Identifies some components; mapping to the artifact is weak or partially incorrect.	Little or no correct identification of architecture/components.	10
Identification of Underlying Technique/Algorithm/Reasoning Mechanism	Correctly reverse-engineers the technique, algorithm, or reasoning	Correctly identifies the technique/mechanism with	Identification is only partially correct or lacks sufficient	Technique/mechanism is incorrectly identified or not addressed.	10

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
	mechanism (e.g., chaining type, ML technique) with clear supporting evidence.	adequate, mostly accurate justification.	supporting evidence.		
Quality of Justification & Use of Evidence (linking conclusions back to the given artifact)	Every conclusion is clearly justified with specific, accurate references to the given artifact.	Most conclusions are justified with reasonable references to the artifact.	Justification is vague or only loosely connected to the artifact.	Conclusions are stated with little or no justification or evidence.	10
Clarity & Completeness of Presentation (addresses all sub-parts of the question in an organised manner)	All sub-parts are addressed completely, in a clear, well-organised, and logically structured manner.	Most sub-parts are addressed clearly with minor gaps in organisation or completeness.	Some sub-parts are missing or the presentation is unclear/poorly organised.	Response is incomplete, disorganised, or largely missing.	10
				Total	50

1Q)

Reverse-Engineering the Flight-Booking Chatbot

(a) PEAS Description

PEAS Component	Description for Flight-Booking Chatbot
P – Performance Measure	Correct flight booking, accurate destination/date/time, suitable flight recommendations, fast response, user satisfaction, low booking errors
E – Environment	Users, airline databases, flight schedules, availability systems, prices, booking/payment systems
A – Actuators	Chat messages, flight recommendations, booking confirmation, prompts/questions, booking API requests
S – Sensors	User's text messages, flight database, airline availability, prices, dates, preferences and booking status

(b) Type of Agent

The chatbot is primarily a Goal-Based Agent, with some characteristics of a Model-Based Agent.

Justification:

- The chatbot has a specific goal: help the user book a flight to Chennai.
- It asks questions to obtain missing information such as departure time and airline preference.
- It uses available flight information to select suitable options.
- It maintains the conversation context, such as the destination (Chennai) and preference (morning flight, no airline preference).

Therefore, it can be classified as a goal-based conversational agent, supported by an internal model/knowledge base of flights.

(c) Component Responsible for Each Chatbot Response

Chatbot Response	Main Component	Reason
“Sure! What time would you like to depart, and do you have a preferred airline?”	Dialogue Manager	Determines that the destination and date are known but departure time and airline preference are missing. It decides what information to ask next.
“I found 3 morning flights to Chennai on Friday.”	Knowledge Base + Dialogue Manager	The Knowledge Base provides flight availability, while the Dialogue Manager uses the user's preferences to select/filter the relevant flights.
“Shall I show you the cheapest option first?”	Dialogue Manager + NLG	The Dialogue Manager decides to offer the cheapest flight, while NLG converts that decision into natural language.

Role of other components:

- NLU (Natural Language Understanding): Converts “*I want to book a flight to Chennai next Friday*” into structured information such as:
 - Intent = Book Flight
 - Destination = Chennai
 - Date = Next Friday
- Dialogue Manager: Maintains conversation state and decides the next action.

- Knowledge Base: Stores/retrieves flight schedules, availability, airlines and prices.
- NLG (Natural Language Generation): Produces human-readable responses.

(d) Enhancement Using a Learning-Agent Architecture

A Learning Agent can be added so that the chatbot improves its recommendations from previous interactions.

Architecture:

User



NLU



Dialogue Manager



Knowledge Base / Flight API



Recommendation



User Feedback



Learning Component



Improved Recommendation Model

For example, if users frequently choose morning flights with the lowest price, the learning component can learn this preference and prioritize cheap morning flights in future conversations.

The learning agent can contain:

Component	Function
Performance Element	Selects and recommends flights
Learning Element	Learns from user choices and feedback
Critic	Evaluates whether recommendations were successful
Problem Generator	Explores alternative recommendations to improve future performance

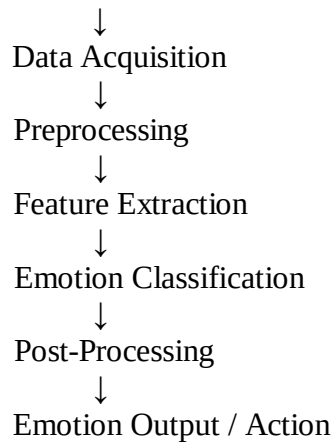
2Q)

Real-Time Emotion Detection System

Since the block diagram is not visible in the message, the following is the standard reverse-engineered pipeline for a real-time emotion detection system.

(a) Function of Each Stage

Input Camera / Microphone

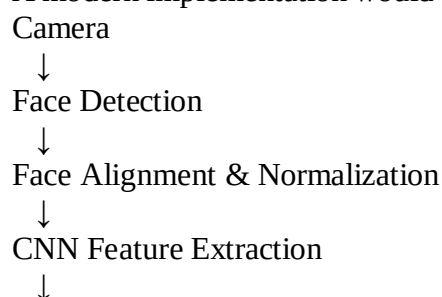


Stage	Function
1. Input / Data Acquisition	Captures real-time facial images, video, or voice from the user.
2. Preprocessing	Removes noise and prepares the input. For images, this may include resizing, grayscale conversion, normalization and face alignment.
3. Feature Extraction	Extracts important characteristics such as facial landmarks, eyes, eyebrows, mouth shape, or speech features.
4. Emotion Classification	Uses the extracted features to determine the user's emotional state such as happy, sad, angry, surprised, fearful or neutral.
5. Post-Processing	Smooths predictions over multiple frames and removes unstable or incorrect predictions.
6. Output / Action	Displays the detected emotion or triggers an appropriate system response.

(b) Likely AI/ML Technique at Each Stage

Stage	Possible AI/ML Technique
Data Acquisition	Computer vision / speech processing
Preprocessing	Image normalization, filtering, face detection using Haar Cascade or MTCNN
Feature Extraction	CNN, HOG, facial landmarks, or deep feature embeddings
Emotion Classification	CNN, SVM, Random Forest, or a pretrained deep-learning model
Post-Processing	Moving average, majority voting, temporal smoothing
Output	Rule-based decision system or intelligent-agent action

A modern implementation would commonly use:



Emotion Classifier



Happy / Sad / Angry / Fear / Surprise / Neutral

(c) PEAS Description

PEAS	Description
P – Performance Measure	Emotion recognition accuracy, low latency, correct classification, robustness to lighting/pose variations and real-time performance
E – Environment	Real-world users, camera/microphone environment, different lighting conditions, backgrounds, facial expressions and speaking conditions
A – Actuators	Display detected emotion, generate alerts, change UI, provide recommendations or trigger an application response
S – Sensors	Camera, microphone, video frames, facial images and audio signals

(d) Agent Type

The system can primarily be classified as a Model-Based Reflex Agent.

Justification:

A simple reflex agent would directly map the current input to an action:

IF angry → show angry

IF happy → show happy

However, a real-time emotion detection system generally maintains information from previous frames to make its prediction more stable.

For example:

Frame 1 → Happy

Frame 2 → Happy

Frame 3 → Neutral

Frame 4 → Happy



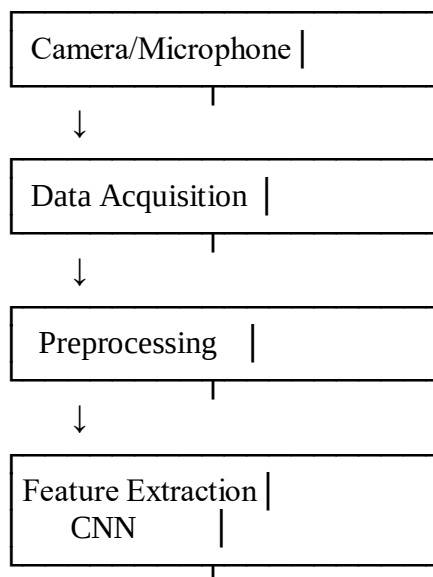
Temporal Processing

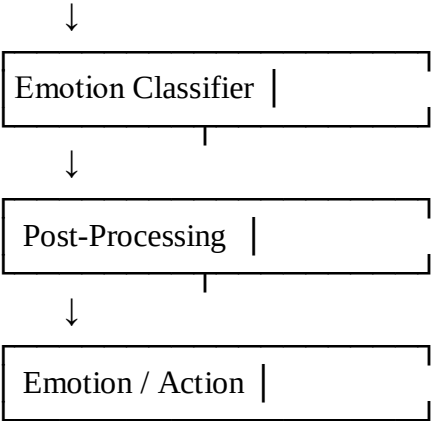


Final Emotion → Happy

Therefore, it uses the current observation together with information about previous observations. This makes Model-Based Reflex Agent the more appropriate classification.

Overall Architecture





3Q)

Expert System Shell for Medical Diagnosis

(a) Type of Chaining Used

The system uses Forward Chaining.

Justification

Forward chaining starts with the given facts and repeatedly applies rules whose conditions are satisfied until a conclusion is reached.

Given facts:

fever = yes

cough = yes

body_ache = yes

The reasoning proceeds as follows:

Given Facts

↓

fever = yes + cough = yes

↓

R1 fires

↓

suspect = flu

↓

suspect = flu + body_ache = yes

↓

R2 fires

↓

diagnosis = Influenza

Therefore, the system starts from available facts and moves forward toward the diagnosis, which is characteristic of forward chaining.

(b) Expert-System Shell Components

Component	Evidence in the Given System	Function
Knowledge Base	R1, R2 and R3	Stores medical knowledge in the form of IF-THEN rules.
Inference Engine	“R1 fires first → R2 then fires”	Matches facts against rule conditions and determines which rules should execute.
Working Memory / Fact Base	fever=yes, cough=yes, body_ache=yes, and derived suspect=flu	Stores current facts and newly inferred facts.
Explanation Facility	“Diagnosis = Influenza because...”	Explains how the system arrived at its diagnosis.
Knowledge Acquisition Module	Used when adding a dengue rule	Allows a human expert to provide new medical knowledge and converts it into rules.

(c) Knowledge Acquisition for Adding a Dengue Rule

To add knowledge about dengue, the knowledge acquisition component would obtain the rule from a medical expert and encode it in the knowledge base.

For example, a medical expert might specify:

IF fever = yes

AND rash = yes

AND body_ache = yes

THEN diagnosis = Dengue

The knowledge-acquisition process would be:

Medical Expert



Provide Dengue Symptoms



Knowledge Acquisition Module



Convert into IF–THEN Rule



Validate Rule



Add to Knowledge Base



Inference Engine can use the new rule

The new rule could be stored as:

Rule IF (Condition)

THEN (Conclusion)

R4 fever = yes AND rash = yes AND body_ache = yes diagnosis = Dengue

The expert system can then use R4 whenever all three conditions are present.

Note: In a real medical system, such a rule should be clinically validated before deployment.

(d) Could Backward Chaining Reach the Same Diagnosis?

Yes. Backward chaining could reach the same diagnosis, provided the system starts with the goal:

diagnosis = Influenza

Backward chaining works from the goal/conclusion backward toward the required facts.

Forward Chaining	Backward Chaining
Starts with known facts	Starts with a goal
Moves toward conclusions	Works backward to required facts
R1 → R2 → Influenza	Influenza → R2 → R1 → facts
Used in the given reasoning trace	Could also reach the same diagnosis

Final Reasoning

Forward:

Facts → R1 → suspect=flu → R2 → Influenza

Backward:

Influenza ← R2 ← suspect=flu ← R1 ← fever & cough



5Q)

Deepfake & Face Authenticity Verification System

(a) Likely Input Features Used by the Model

The system appears to use facial and temporal features extracted from each video frame.

Feature	Purpose
Blink Rate	Measures natural eye blinking behavior. Abnormal or inconsistent blinking can indicate synthetic/manipulated video.
Texture/Artifact Score	Measures visual inconsistencies such as unnatural skin texture, blending artifacts, facial boundaries, compression patterns, or generated-image artifacts.
Facial Features	Likely includes eyes, mouth, facial landmarks, symmetry and facial movements.
Temporal Features	Compares facial changes between consecutive frames to detect unnatural motion or inconsistencies.

From the table, the texture/artifact score appears particularly important because higher values are associated with higher fake confidence.

For example:

Frame 1 → Texture = 0.12 → Fake confidence = 8%

Frame 3 → Texture = 0.83 → Fake confidence = 91%

(b) Likely Model/Technique Used

The system is likely using a supervised machine-learning/deep-learning classifier.

A possible architecture is:

Video Frame



Face Detection



Feature Extraction



Blink + Texture/Artifact Features



ML/DL Classifier



Fake Confidence Score



Real / Fake

(c) How the Final Real/Fake Decision Was Derived

The system appears to use a majority-voting approach across the five frames.

Assume the decision threshold is approximately 50%:

Frame Fake Confidence Frame Prediction

1	8%	Real
2	74%	Fake
3	91%	Fake
4	5%	Real
5	68%	Fake

Therefore:

Real = 2 frames

Fake = 3 frames

The majority is:

Fake: $3/5 = 60\%$

Real: $2/5 = 40\%$

Hence:

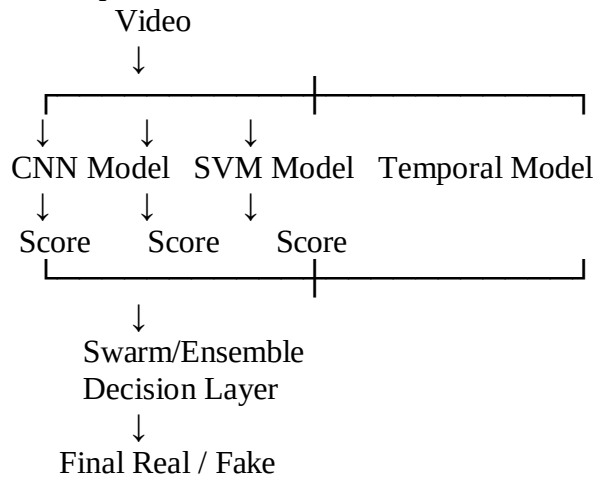
Overall Verdict = FAKE

The important point is that the system does not necessarily require every frame to be fake. A majority of frame-level predictions can determine the final video-level classification.

(d) Swarm/Ensemble-Based Improvement

A swarm/ensemble approach could improve reliability by using several independent detection models instead of relying on one classifier.

For example:



Each model could specialize in a different type of evidence:

- CNN Agent: Detects visual manipulation artifacts.
- Blink Agent: Checks eye-blinking patterns.
- Texture Agent: Detects unnatural skin/face texture.
- Temporal Agent: Detects inconsistent movement between frames.
- Audio Agent: Checks whether audio and lip movement are synchronized.

The swarm/ensemble could combine their confidence scores using weighted voting. More reliable models would receive higher weights.

Example

CNN → Fake 90%

Texture → Fake 85%

Blink → Real 60%

Temporal → Fake 88%

Audio → Fake 82%

↓
Ensemble Decision

↓
FAKE (high confidence)

This is more reliable than depending on a single feature because multiple independent detectors must agree before producing the final decision.